

Arquitectura en Capas Distribuidas

Sistema de Tareas – Arquitectura de Software

Curso de Arquitectura de Software

Agenda

- Escenario y ADR 001 – Justificación
- C4 Nivel 2 – Diagrama de Contenedores
- C4 Nivel 3 – Diagrama de Componentes
- Función de cada capa (4 capas lógicas)
- Flujo de ejemplo: cambio de estado
- Resumen y patrones aplicados

Escenario y Requerimientos

- Plataforma de gestión de trabajo para equipos distribuidos
- **100 a 10,000 usuarios concurrentes**
- CRUD de tareas con ciclo de vida por estados
- Notificaciones por email y SMS
- Reportes agregados de productividad
- Requerimientos no funcionales:
 - Consultas < 200ms
 - Alta disponibilidad en horario laboral
 - Separación de responsabilidades por área operativa

ADR 001 – Decisión

Arquitectura en capas distribuidas (N-tier) con 3 tiers físicos:

1. **Tier de Presentación** – SPA React + Nginx (expuesto a internet)
2. **Tier de Aplicación + Dominio** – API Spring Boot (subred protegida)
3. **Tier de Infraestructura** – PostgreSQL + servicios externos (subred de datos)

Comunicación entre tiers: **REST (HTTP/JSON)**

Dependencias internas: regla **DIP** – siempre hacia el Dominio

Alternativas Consideradas

Monolito en Capas

- Simplicidad operativa
- Transacciones directas
- Un solo despliegue Descartado:
- No escala frontend/backend por separado
- Sin aislamiento DMZ

Microservicios

- Escalado granular
- Despliegue independiente
- Equipos autónomos Descartado:
- Sobre-ingeniería para dominio acotado
- Complejidad operativa injustificada

ADR – Consecuencias

Impacto positivo:

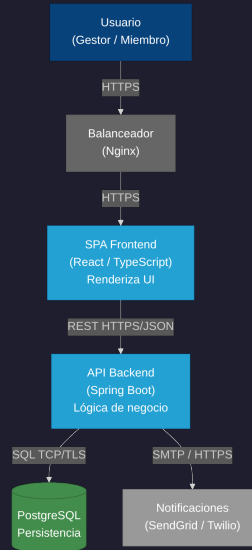
- Escalado independiente del frontend
- Seguridad por zona (DMZ)
- Mantenibilidad por equipo (frontend, backend, DBA)
- Separación clara de responsabilidades

Impacto negativo:

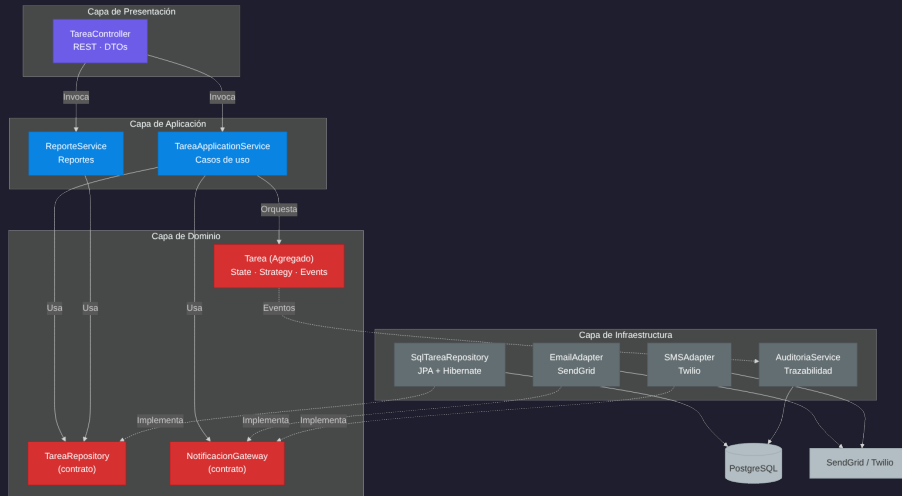
- Latencia de red entre tiers
- Fallos parciales (requiere reintentos)
- Complejidad operativa (3 superficies de monitoreo)

Trade-off aceptado: la ganancia en escalabilidad, seguridad y organización de equipos justifica el costo en latencia y complejidad.

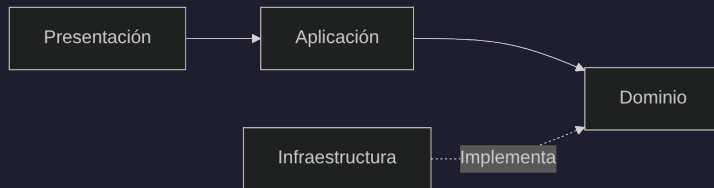
C4 Nivel 2 – Diagrama de Contenedores



C4 Nivel 3 – Componentes del Backend



Dirección de Dependencias (DIP)



- Las dependencias siempre apuntan hacia el Dominio.
- La Infraestructura implementa contratos definidos por el Dominio.
- El Dominio no conoce frameworks, ORMs ni proveedores externos.

Capa de Presentación

Responsabilidad: Traducir protocolos y experiencia de usuario. No decide reglas de negocio.

Componentes:

- `TareaController` – Endpoints REST
 - `GET /tareas`, `POST /tareas`, `PATCH /tareas/{id}/estado`
 - Recibe DTOs, valida formato, delega al servicio de aplicación

Patrones aplicados:

- **Front Controller** – `DispatcherServlet` de Spring
- **DTO / ViewModel** – `TareaRequest`, `TareaResponse`

Lo que NO hace: No contiene lógica de negocio, no accede a la BD, no decide transiciones de estado.

Capa de Aplicación

Responsabilidad: Coordinar los pasos de cada caso de uso. Orquesta, no contiene lógica de negocio.

Componentes:

- `TareaApplicationService`
 - Carga la entidad desde el repositorio
 - Resuelve la estrategia de asignación
 - Delega el cambio de estado al dominio
 - Persiste y dispara notificaciones
- `ReporteService` – Consultas agregadas

Patrones aplicados:

- **Use Case / Application Service**
- **Facade** – API cohesiva hacia presentación

Capa de Dominio – Reglas de Negocio

Responsabilidad: Contener la política y las reglas de negocio. Es el núcleo del sistema. No depende de nada externo.

Componentes principales:

- **Tarea** – Agregado raíz
 - Encapsula ciclo de vida: Pendiente → EnProgreso → Completada/Cancelada
 - Contiene: título, descripción, prioridad, fecha límite, responsable, estado
- **EstadoTarea** – State Pattern
 - Pendiente → EnProgreso o Cancelada
 - EnProgreso → Completada o Cancelada
 - Completada y Cancelada → estados terminales
- **TareaEventoDominio** – Domain Events
 - **TareaCreada**, **TareaAsignada**, **TareaCompletada**

Capa de Dominio – Patrones

- **Aggregate (DDD):** `Tarea` es raíz, garantiza consistencia interna
- **State:** Cada estado es una clase; `Tarea` delega transiciones
- **Strategy:** `AsignacionPolicy` – ¿quién puede asignar?
 - Líder: asigna a cualquier miembro
 - Miembro: solo auto-asignarse tareas libres
- **Strategy:** `SLAStrategy` – prioridad dinámica por tiempo restante
- **Domain Events:** Desacoplan reacciones del flujo principal
- **Interfaces (contratos):** `TareaRepository`, `NotificacionGateway`
 - Definidas aquí, implementadas en infraestructura

Capa de Infraestructura

Responsabilidad: Implementar persistencia e integraciones externas. Interactúa con el mundo exterior.

Componentes:

- `SqlTareaRepository` – JPA + Hibernate → PostgreSQL Implementa `TareaRepository` (contrato del dominio)
- `EmailAdapter` – SendGrid (SMTP/API) Implementa `NotificacionGateway`
- `SMSAdapter` – Twilio (API REST) Implementa `NotificacionGateway`
- `AuditoriaService` – Escucha eventos de dominio Registra trazabilidad de cambios en tabla `auditoria_tareas`

Patrones aplicados: Repository, Data Mapper (ORM), Adapter/Gateway, Event Listener

Flujo de Ejemplo – Cambio de Estado

PATCH /tareas/42/estado → marcar como COMPLETADA

Paso 1 – Presentación: `TareaController` recibe la petición, valida el DTO, invoca al servicio.

Paso 2 – Aplicación: `TareaApplicationService` carga la `Tarea` desde el repositorio.

Paso 3 – Dominio: `Tarea` delega en `EstadoEnProgreso.completar()`. Valida la transición, muta a `Completada`, emite `TareaCompletadaEvent`.

Paso 4 – Aplicación: Persiste con `TareaRepository.save(tarea)`.

Paso 5 – Infraestructura: `SqlTareaRepository` ejecuta `UPDATE tareas SET estado = 'COMPLETADA' WHERE id = 42`.

Paso 6 – Aplicación: Notifica: `NotificacionGateway.notificarCambioEstado(tarea)`.

Paso 7 – Infraestructura: `EmailAdapter` envía el correo vía SendGrid.

Paso 8 – Infraestructura (async): `AuditoriaService` recibe el evento y registra auditoría.

Resumen de Capas

- **Presentación** – Traduce HTTP/UI Patrones: Front Controller, DTO → Depende de Aplicación
- **Aplicación** – Coordina casos de uso Patrones: Use Case, Facade → Depende de Dominio (contratos)
- **Dominio** – Contiene reglas de negocio Patrones: Aggregate, State, Strategy, Domain Events → No depende de nada externo
- **Infraestructura** – Implementa persistencia e integraciones Patrones: Repository, Adapter, ORM → Implementa contratos del Dominio

DIP: Presentación → Aplicación → Dominio ← Infraestructura